



TECNOLÓGICO
NACIONAL DE MÉXICO



Tecnológico Nacional de México

Campus Saltillo

Ingeniería en Sistemas Computacionales

Computo en la nube

4.2 Demostrar la inyección SQL

Docente: Ing. Miguel Salazar del Bosque

Alumno:

Jesus Contreras Castillo 22050648

- 1- Crear la base de datos **vulnerable_db**, y dentro de ésta crear la tabla **usuarios** y meter 2 registros de prueba

```
jesusc@SRVREP:~$ mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.8.3-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE vulnerable_db;
Query OK, 1 row affected (0.002 sec)

MariaDB [(none)]> USE vulnerable_db;
Database changed
MariaDB [vulnerable_db]> CREATE TABLE usuarios (
  ->   id INT AUTO_INCREMENT PRIMARY KEY,
  ->   nombre_usuario VARCHAR(50),
  ->   contrasena VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.025 sec)

MariaDB [vulnerable_db]> INSERT INTO usuarios (nombre_usuario, contrasena) VALUES (
'admin', '1234');
Query OK, 1 row affected (0.004 sec)

MariaDB [vulnerable_db]> INSERT INTO usuarios (nombre_usuario, contrasena) VALUES (
'user', 'password');
Query OK, 1 row affected (0.002 sec)

MariaDB [vulnerable_db]> |
```

- 2- Probar vulnerabilidad:

Se puede ingresar con el usuario y contraseña regular

Login exitoso

Usuario:

Contraseña:

De igual forma se puede ingresar usando el usuario sin contraseña ingresando:
admin' -- sin contraseña:

Login exitoso

Usuario:

Contraseña:

3- Corregir la vulnerabilidad:

```
#!/usr/bin/php
$conn = new mysqli('localhost', 'webuser', '1234', 'vulnerable_db');

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $usuario = $_POST['usuario'];
    $contrasena = $_POST['contrasena'];

    $sql = "SELECT * FROM usuarios WHERE nombre_usuario='$usuario' AND contrasena='$contrasena'";
    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        echo "Login exitoso";
    } else {
        echo "Credenciales inválidas";
    }
}

?>

<form method="POST">
    Usuario: <input type="text" name="usuario"><br>
    Contraseña: <input type="password" name="contrasena"><br>
    <input type="submit" value="Ingresar">
</form>
```

Línea que causa
la vulnerabilidad

Corregido usando bind_param:

```
GNU nano 8.4 /var/www/html/login.php *
<?php
ini_set('display_errors', 1);
error_reporting(E_ALL);

$conn = new mysqli('localhost', 'webuser', '1234', 'vulnerable_db');

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $usuario = $_POST['usuario'];
    $contrasena = $_POST['contrasena'];

    // Consulta segura
    $stmt = $conn->prepare("SELECT * FROM usuarios WHERE nombre_usuario=? AND contrasena=?");
    $stmt->bind_param("ss", $usuario, $contrasena);
    $stmt->execute();

    $result = $stmt->get_result();

    if ($result && $result->num_rows > 0) {
        echo "Login exitoso";
    } else {
        echo "Credenciales inválidas";
    }
}

?>

<form method="POST">
    Usuario: <input type="text" name="usuario"><br>
    Contraseña: <input type="password" name="contrasena"><br>
    <input type="submit" value="Ingresar">
</form>
```

4- Volvemos a verificar la vulnerabilidad:

Podemos ingresar con el usuario normal:

Login exitoso

Usuario:

Contraseña:

Deniega el acceso si tratamos de hacer inyección sql:

Credenciales inválidas

Usuario:

Contraseña: